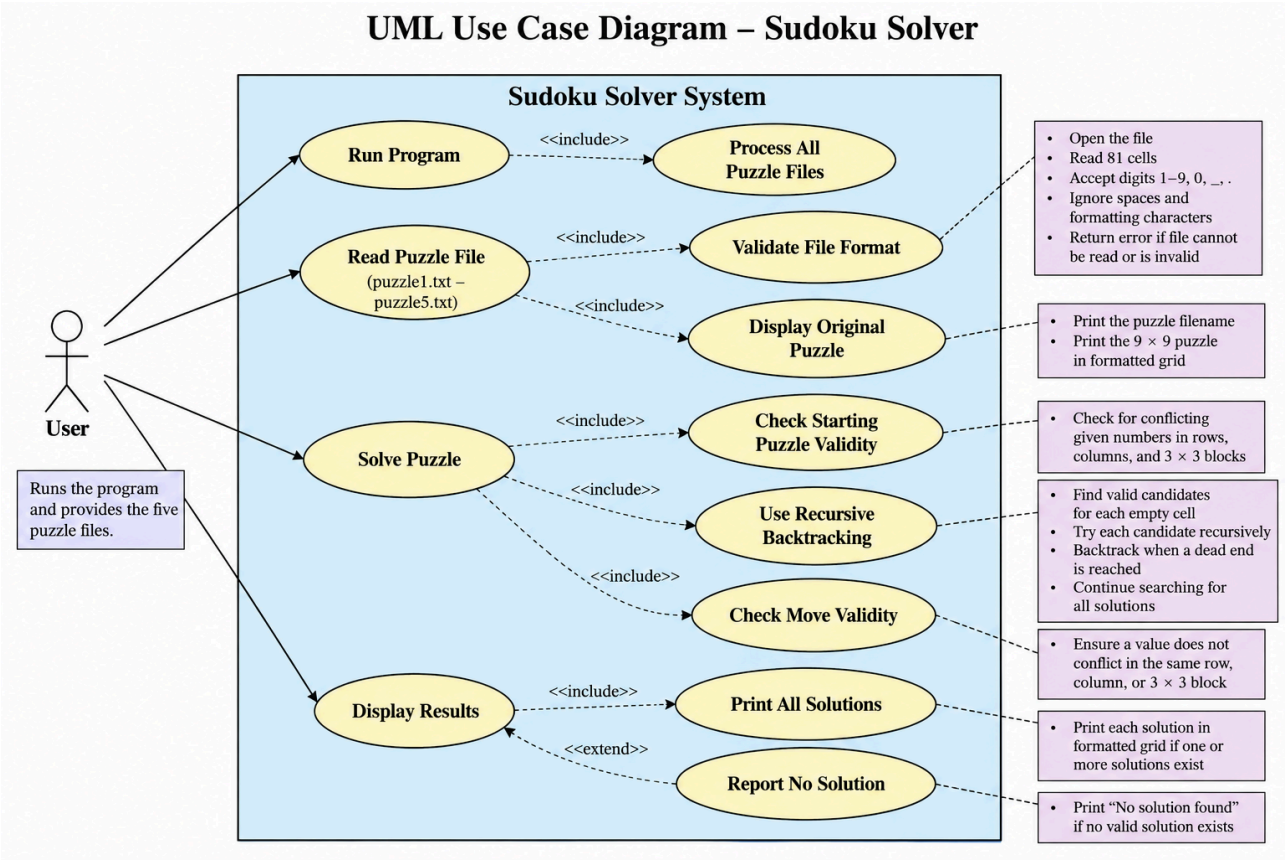


UML Use Case Model – Sudoku Solver

1. Identify/Discover Use Cases

Actor: User

System: Sudoku Solver System

Use Cases:

- Run Program
- Process All Puzzle Files
- Read Puzzle File
- Validate File Format
- Display Original Puzzle
- Solve Puzzle
- Check Starting Puzzle Validity
- Use Recursive Backtracking
- Check Move Validity
- Display Results
- Print All Solutions
- Report No Solution

The primary actor is the User because the user starts the program and receives the puzzle results. The Sudoku Solver System processes the five provided puzzle files and either displays all valid solutions or reports that no solution exists.

2. Brief Descriptions

Run Program: Allows the user to start the Sudoku solver so that the five puzzle files can be processed.

Process All Puzzle Files: Allows the system to process puzzle1.txt through puzzle5.txt in order.

Read Puzzle File: Opens the current puzzle file and reads the values used to create the 9x9 Sudoku grid.

Validate File Format: Checks that enough valid values or blank markers were read to form a complete Sudoku puzzle.

Display Original Puzzle: Displays the name of the puzzle file and the original puzzle before solving it.

Solve Puzzle: Searches for all valid solutions to the current Sudoku puzzle.

Check Starting Puzzle Validity: Checks whether the original numbers in the puzzle conflict with another number in the same row, column, or 3x3 block.

Use Recursive Backtracking: Tries possible values in empty cells and goes back to try a different value when a choice leads to a dead end.

Check Move Validity: Determines whether a possible value can legally be placed in an empty cell according to Sudoku rules.

Display Results: Shows the result of attempting to solve the current puzzle.

Print All Solutions: Displays every valid solution found for the puzzle.

Report No Solution: Displays “No solution found” when the puzzle cannot be completed successfully.

3. Step-by-Step Flow of Events

Use Case: Process All Puzzle Files

1. The user starts the Sudoku Solver program.
2. The system selects the first puzzle file.
3. The system opens and reads the puzzle file.
4. The system stores the given numbers and blank spaces in a 9x9 grid.
5. The system checks that a complete puzzle was successfully read.

6. The system displays the puzzle filename and original puzzle.
7. The system checks whether the starting values follow Sudoku rules.
8. The system locates an empty cell.
9. The system determines which values from 1 through 9 can legally be placed in the empty cell.
10. The system places a valid candidate into the cell.
11. The system recursively continues the same process for the remaining empty cells.
12. If a candidate leads to a dead end, the system removes that value and tries another candidate.
13. When all cells are filled correctly, the system records and displays the completed solution.
14. The system continues searching to determine whether more solutions exist.
15. If no valid solution is found, the system displays “No solution found.”
16. The system moves on to the next puzzle file.
17. The process repeats until all five puzzle files have been processed.
18. The program ends.

4. Detailed Use Case Description

Use Case: Process All Puzzle Files

Primary Actor: User

Goal: The user wants the Sudoku Solver System to process the five provided Sudoku puzzles and display all valid solutions for each puzzle.

Preconditions:

- The Sudoku Solver program is able to run.
- The puzzle files puzzle1.txt through puzzle5.txt are available to the program.
- Each file is intended to contain a 9x9 Sudoku puzzle.

Trigger:
The use case begins when the user runs the Sudoku Solver program.

Basic Flow:

1. Start Program
   The user runs the Sudoku Solver program. The system begins with puzzle1.txt and processes the puzzle files in numerical order.
2. Read Puzzle File
   The system opens the current puzzle file and reads its contents. Numbers 1 through 9 are stored as filled cells, while blank markers are stored as empty cells.

3. Validate Puzzle Input
   The system verifies that enough information was read to create a complete 9x9 Sudoku grid. If the file cannot be read correctly, the system reports an error and moves to the next puzzle.
4. Display Original Puzzle
   The system displays the name of the current puzzle file and prints the original unsolved puzzle.
5. Check Starting Puzzle
   The system checks the given values to make sure the original puzzle does not already violate Sudoku rules. Each given number must not conflict with another number in the same row, column, or 3x3 block.
6. Find an Empty Cell
   The system searches the puzzle for an empty cell. If one is found, the solving process continues. If there are no empty cells remaining, the current grid represents a complete solution.
7. Determine Valid Candidates
   The system checks the values 1 through 9 for the selected empty cell. A value is valid only if it does not already appear in the same row, column, or 3x3 block.
8. Try a Candidate
   The system places one valid candidate into the empty cell and recursively attempts to solve the rest of the puzzle.
9. Backtrack if Necessary
   If the selected candidate eventually causes a dead end, the system removes that value and tries the next valid candidate. This continues until a solution is found or every possible candidate has been tested.
10. Display a Solution
    When all cells have been successfully filled, the system counts the completed grid as a solution and displays it to the user.
11. Search for Additional Solutions
    The system continues the backtracking search after finding a solution so that it can determine whether the puzzle has any additional valid solutions. The assignment specifically requires all solutions to be printed when a puzzle has more than one solution. Assignment 4 Instructions
12. Process the Next File
    Once all possible solutions for the current puzzle have been found, the system moves to the next puzzle file and repeats the process.
13. End Program
    After puzzle5.txt has been processed, the program finishes.

Alternative Flow 1: File Cannot Be Read

If the system cannot open or correctly read a puzzle file, it displays an error message for that file and continues to the next puzzle.

Alternative Flow 2: Invalid Starting Puzzle

If the original puzzle contains conflicting given numbers, the system determines that the puzzle cannot be solved and reports that no solution was found.

Alternative Flow 3: No Solution Exists

If the recursive search tries every possible valid candidate and no complete Sudoku grid can be created, the system displays “No solution found.”

Alternative Flow 4: Multiple Solutions Exist

If a complete solution is found but other possible candidate combinations still exist, the system continues searching. Each additional valid solution is displayed until every possible solution has been found.

Postconditions:

- The original puzzle has been displayed.
- All valid solutions have been displayed if solutions exist.
- If no solution exists, the user has been informed.
- The system continues until all five puzzle files have been processed.
- The original puzzle files are not changed.